Project 3 Documentation

> **Team 31**

Github Repository Link : https://github.com/paridhii16/SE-Project-3

Task 1: Requirements and Subsystems

The StayFit system helps health enthusiasts and dieters by providing AI-assisted meal logging, real-time calorie tracking, and personalized nutritional recommendations to achieve their fitness goals.

Functional Requirements

User Authentication & Profile Management: Handles user authentication via native secure email/password registration using `werkzeug.security` for hashing, and SSO integration with Google OAuth2 Federated Identity. It issues JSON Web Tokens (`flask_jwt_extended`) with a 7-day expiration (`JWT_ACCESS_TOKEN_EXPIRES`) for session management. It manages comprehensive dynamic user profiles (weight, height, age, gender, 5-tier activity level) to calculate personalized daily calorie budgets using the **Mifflin-St Jeor Equation** within `calorie_service.py` (adjusting by +/- 500 calories based on Lose/Maintain/Gain goals)

Architectural Significance

OAuth2 and JWT authentication require the API routing subsystem to support stateless auth and verify tokens per request, acting as a gateway for all protected routes. The dynamic profile engine shapes the core domain logic, as the BMR and TDEE calculations dictate the user's entire experience, goal tracking, and AI recommendation thresholds throughout the system.

AI-Assisted Meal Logging: Enables users to upload food images (JPEG/PNG) or provide natural language text descriptions. Inside `tasks.py`, high-resolution image uploads are automatically resized to **512×512 thumbnails** and re-encoded to JPEG (85% quality), reducing payload size by ~90%. The system prompts the multimodal AI to return a strictly structured JSON object extracting individual items, portions, and precision macros (Calories, Protein, Carbs, Fat, Fiber).

Architectural Significance

This forms the core value proposition of the system. It necessitates an asynchronous background processing pipeline (Celery with a Redis broker) to prevent long-running AI API calls (5-15s) from blocking the web server. It also introduces the need for the Strategy design pattern (housed in `AIProviderStrategy`) to ensure high availability via a `ProviderChain`, automatically falling back from the primary `Gemini 2.5 Flash` model to `Llama 3/4` via Groq if unresponsive or rate-limited.

Calorie Tracking & Analytics: The `DailyLogRepository` maintains a running tally of a user's nutritional intake, triggering atomic updates for every new meal logged. It categorizes meals (Breakfast, Lunch, Dinner, Snacks) and powers the `analytics` subsystem which calculates "Weekly Trends" by aggregating daily logs over the past 7 days for the frontend progress charts.

Architectural Significance

Drives the dashboard UI and requires highly optimized database access. To prevent straining the PostgreSQL database during heavy iterative queries for aggregations on every page navigation, it necessitates an in-memory Redis caching layer (via `cache_service.py`) and cache-aside patterns to serve view payloads with minimal latency.

AI Meal Recommendations: Generates context-aware recipe suggestions (3 specific meal options) tailored to the user's `remaining_calories` for the day and specific dietary goals (e.g., "High Protein"), complete with preparation steps and nutritional highlights.

Architectural Significance

Requires seamless integration with LLMs and prompts engineered for strict JSON output. It heavily influences the caching strategy, as semantic caching is required to share identical AI-generated recommendations among users with identical calorie/goal constraints, drastically reducing external API token costs and generation latency.

Smart Notifications: Provides real-time event streams via the `/stream` endpoint (Server-Sent Events) to push updates to the browser without client refreshing. It uses **Redis Pub/Sub** to broadcast signals on channels like `meal_analysis` and `calorie_alerts`, filtering them so users only receive updates for their own `user_id`.

Architectural Significance

Introduces an event-driven Observer pattern publish-subscribe architecture. By transitioning from Postgres `LISTEN/NOTIFY` to Redis Pub/Sub, it alleviates the database from maintaining long-lived idle connections. This demands a unidirectional stream (SSE) to push updates from the backend to the frontend, preventing the overhead of thousands of clients polling the API and drastically reducing server CPU and database load.

Non-Functional Requirements

1. Performance (Asynchronous Processing & Caching) Performance requirements dictated the separation of fast HTTP request handling from slow I/O-bound AI tasks, and required minimizing database load for frequent read operations. These requirements shaped:

- The adoption of Celery and a Redis broker for background task queueing, returning a `task_id` and "202 Accepted" immediately so the web server is never blocked by AI latency.
- The implementation of a Redis Caching Layer (`@cache_response` decorator) to memoize expensive analytical aggregations and provide caching for meal recommendations, reducing dashboard load times to 1-3ms and preventing redundant API calls.
- Automated image optimization (512×512/85%) on the backend to reduce AI payload size and network transit time.

2. Availability (Fault Tolerance & Active Redundancy) Availability requirements resulted in a highly resilient architecture that can survive third-party outages, as the application's core feature relies on external AI providers. These requirements shaped:

- The implementation of the Strategy Design Pattern combined with the `tenacity` library for exponential backoff retries.

- The `ProviderChain` which handles service-level exceptions and seamlessly switches the "Strategy" from Gemini 2.5 Flash to Groq (Llama) models if rate limits or downtimes occur.
- Exception logging modules to isolate external failures from crashing the core platform.

3. Modifiability (Layered Architecture & Microservices) The platform is designed for maintainability, testability, and independent scaling of components. This requirement led to:

- The transition to a Microservices Architecture (via `start_microservices.py`), decoupling the API HTTP servers, background Celery workers, and notification dispatchers into independently scalable processes.
- Strict enforcement of an MVC/Layered structure utilizing Flask Blueprints (auth, meals, analytics, notifications) to organize routes.
- Separation of concerns through a modular design utilizing the Repository Pattern, where routes handle HTTP/JWT, services handle business logic (e.g., `calorie_service`), and repositories abstract raw SQL interactions with SQLAlchemy models, facilitating unit testing and preventing tightly coupled spaghetti code.

4. Usability (Real-time Feedback & Accessibility) User responsiveness is critical to masking the inherent delays of AI processing. The user must feel the system is fast and responsive across all devices, even mobile. These requirements shaped:

- The use of Server-Sent Events (SSE) and a React Context API (`MealContext`) for optimistic UI updates, ensuring the dashboard re-renders automatically the moment an asynchronous task finishes without a page refresh.
- The implementation of a Progressive Web App (PWA) architecture with Service Workers and a Web App Manifest (`PWA_README.md`), allowing offline asset caching and a native-app-like installation experience on mobile home screens.

5. Security (Authentication & Data Privacy) Security requirements mandated the protection of user health data and robust access control. These requirements shaped:

- The implementation of Federated Identity (Google OAuth2) to offload password storage and security risks.
- The use of JSON Web Tokens (JWT) for stateless session management and Role-Based Access Control, ensuring users can only access their own isolated database records.

6. Testability (Isolation & Mocking) Testability requirements dictated that the system must be easily verifiable without requiring full end-to-end setups. These requirements shaped:

- The adoption of the Repository and Strategy patterns, allowing database calls and external AI API integrations to be easily mocked during unit testing.
- The separation of core business logic into decoupled services that can be tested independently of the Flask routing and HTTP request context.

7. Observability (Monitoring & Logging) The system must provide deep visibility into production errors and asynchronous task failures to ensure quick resolution. These requirements shaped:

- The integration of error tracking tools (like Sentry) to capture stack traces and context for both the Flask web server and background Celery workers.

Subsystem Overview

1. Frontend Presentation Subsystem

- **Role:** Provides the user interface and handles local state management across desktop and mobile.
- **Functionality:** Displays dashboards, meal logs, and recommendations using a premium React/Vite CSS design system, Leverages the React Context API to instantly reflect real-time updates received via SSE, Acts as a Progressive Web App (PWA) to provide native installation and offline caching capabilities.

2. API Routing & Authentication Subsystem

- **Role:** Acts as the primary HTTP entry point and secures the application.
- **Functionality:** Manages user login/registration via OAuth2 (Google Federated Identity) and native credentials, Issues and validates JWT tokens, Acts as a gateway, enforcing access control and routing requests to the appropriate internal services.

3. Business Logic & AI Services Subsystem

- **Role:** Executes domain rules and orchestrates external AI intelligence.
- **Functionality:** Calculates BMR, TDEE, and macro-splits, Implements the Strategy Pattern to manage the AI `ProviderChain`, handling prompt engineering, image encoding, and JSON extraction from Gemini and Groq APIs.

4. Data Access Subsystem

- **Role:** Manages all interactions with persistent storage.
- **Functionality:** Employs the Repository Pattern to abstract raw SQL/ORM commands away from business services, Ensures data consistency when logging meals and retrieving aggregated daily or weekly logs.

5. Real-time Notification Subsystem

- **Role:** Delivers asynchronous event updates to connected clients.
- **Functionality:** Listens to PostgreSQL pub/sub channels (`LISTEN/NOTIFY`) for task completion signals and streams them directly to the React frontend via Server-Sent Events (SSE).

Supporting Service Components

1. **Caching Service**

- **Role:** Accelerates data retrieval and minimizes redundant external API calls.
- **Functionality:** Uses `cache_service.py` to implement a Cache-Aside pattern via custom decorators, storing dashboard analytics aggregations and semantically identical AI recommendation outputs in Redis for up to 5 minutes.

2. **Asynchronous Worker Service (Celery)**

- **Role:** Offloads heavy computation and network waiting from the main web server.
- **Functionality:** Independent worker processes that pull jobs from a Redis queue. They handle the "CPU bound" tasks of image processing via the `PIL` (Pillow) library and the "I/O bound" tasks of communicating with Gemini/Groq APIs, ultimately inserting parsed results into the database.

Supporting Infrastructure

- **Role:** Provides foundational storage, messaging, and orchestration for the system.
- **Functionality:** Uses **PostgreSQL** for persistent relational storage, **Redis** acting centrally as a high-speed message broker for Celery, an in-memory data cache, and the core Pub/Sub event broadcasting engine, and **Microservices Orchestration scripts** (`start_microservices.py`) to launch, decouple, and scale the Python backend processes.

Task 2: Architecture Framework

We have used the **IEEE 42010** standard for describing architecture through stakeholders, concerns, viewpoints, and views.

Part 1: IEEE 42010 Architecture Description

1. Stakeholders

Stakeholders are individuals, teams, or organizations with an interest in, or concerns relative to, the system.

Stakeholder	Description
End Users	Individuals using the application to log meals, track calories, and manage their health goals.
Developers	The software engineering team responsible for building, maintaining, and extending the system.
Instructor/Evaluator	Academic staff assessing the project's adherence to software engineering principles, design patterns, and architectural robustness.
Admin/Operator	Individuals responsible for deploying the system, monitoring uptime, and managing the database and message brokers.
AI/API Providers	Third-party services (Google Gemini, Groq) that provide the core intelligence for image and text analysis.
Security/Privacy Stakeholder	Representatives ensuring that sensitive user health data and passwords are securely stored and transmitted.

2. Concerns

Concerns are the interests of the stakeholders pertaining to the system's development, operation, or quality attributes (Non-Functional Requirements).

Concern	Description	Primary Stakeholders
Usability	The application must be intuitive, providing a seamless meal logging experience without UI freezing.	End Users, Evaluator
AI Accuracy	Nutritional extraction from images and text must be reasonably accurate to be useful.	End Users, Evaluator
Performance	The system must remain responsive, especially during heavy AI processing tasks.	End Users, Developers
Availability	The meal logging feature must remain functional even if external dependencies experience downtime.	End Users, Admin/Operator
Privacy/Security	User authentication must be secure, and health data must be isolated per user.	End Users, Security Stakeholder
Maintainability	The codebase must be modular, layered, and easy to test and extend.	Developers, Evaluator
Scalability	The system architecture must be able to handle multiple users simultaneously executing async tasks.	Admin/Operator, Developers

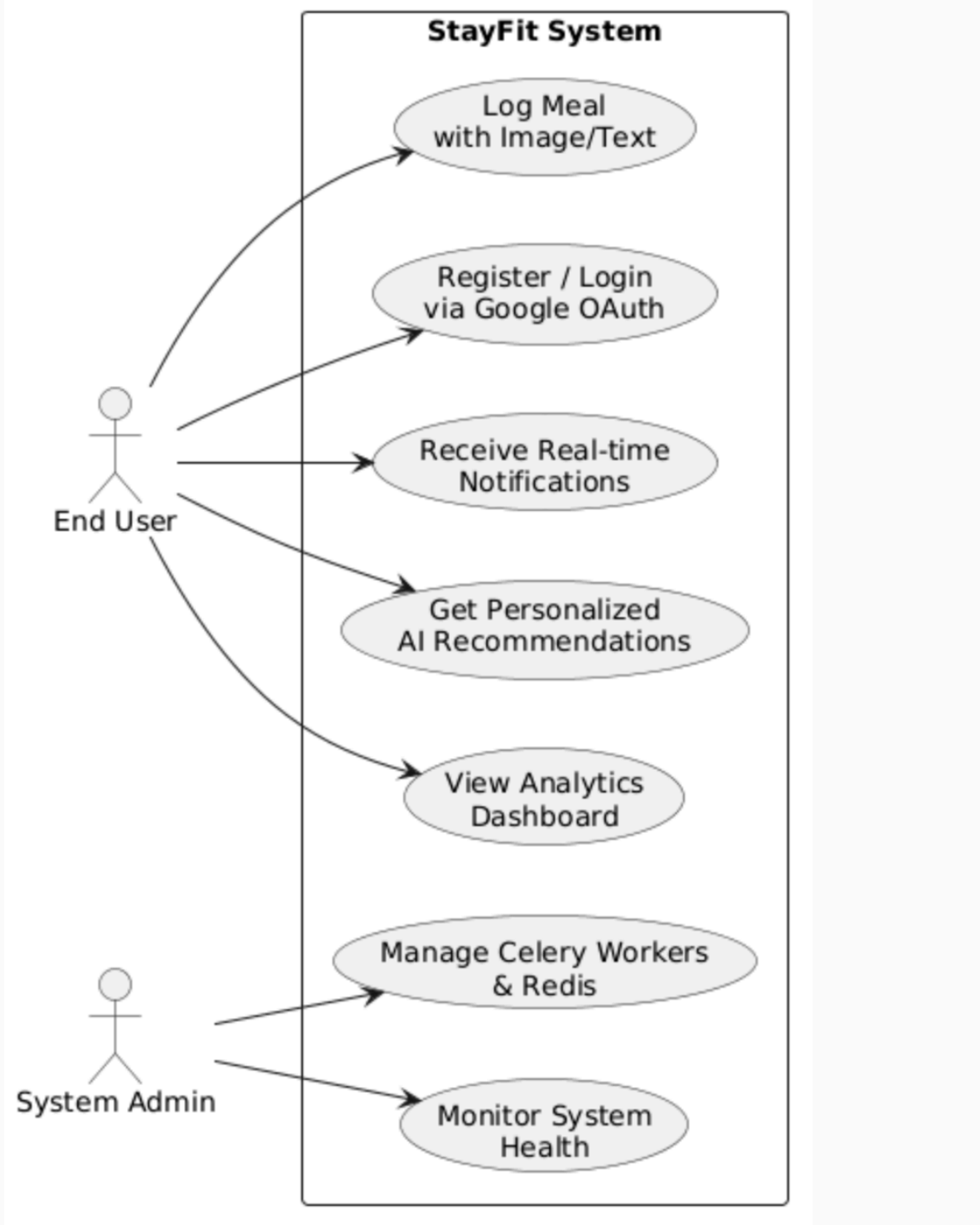
3. Viewpoints and Views Mapping

To address the concerns of the various stakeholders, the architecture is described from multiple viewpoints. Each viewpoint provides a specific "view" into the system.

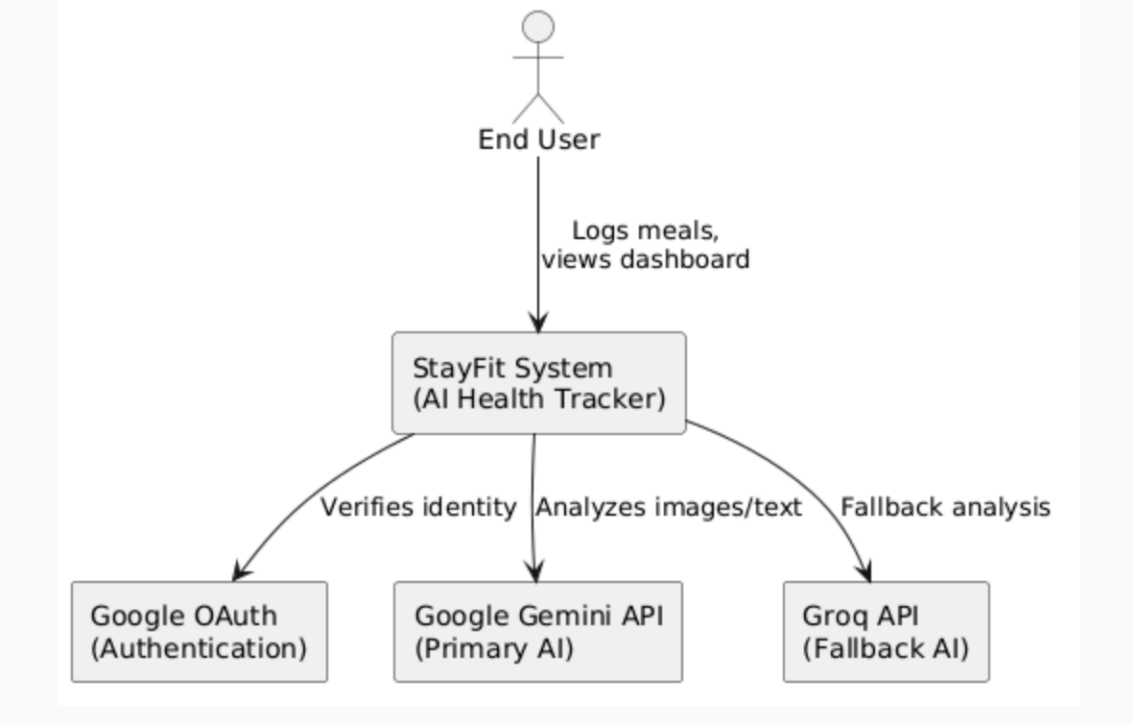
Viewpoint / View	Description & System Mapping	Concerns Addressed
Context View	Describes the system's boundaries and interactions with external entities. *View: Shows the StayFit Backend communicating with the React Frontend, Google Gemini API, and Groq API.*	Usability, AI Accuracy, Privacy/Security
Module View	Describes the static structure of the codebase. *View: Shows the MVC layered architecture (Flask Blueprints → Services → Repositories → Models).*	Maintainability
Runtime/Process View	Describes the dynamic execution and concurrency of the system. *View: Shows the HTTP request flow, the handoff to Celery/Redis for background processing, and the SSE notification loop.*	Performance, Scalability
Data View	Describes the storage, structure, and management of information. *View: Shows the PostgreSQL ERD (Users, Meals, DailyLogs) and how logs aggregate into totals.*	Privacy/Security, Maintainability
Deployment View	Describes the mapping of software components to hardware/infrastructure. *View: Shows the Web Server, Redis Server, Celery Workers, and PostgreSQL Database running as distinct services.*	Availability, Scalability

4. Architectural Diagrams

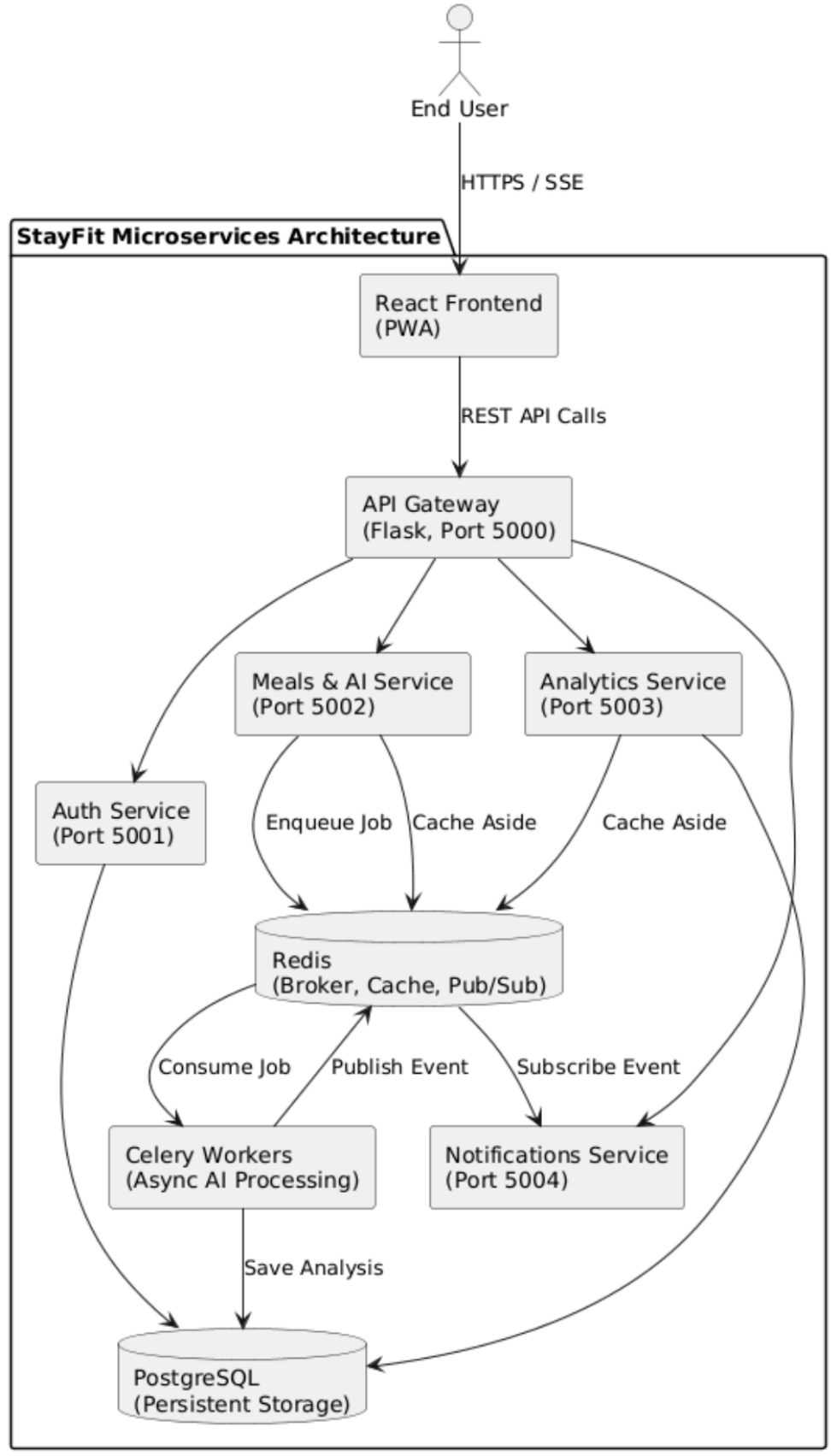
Use Case Diagram (Functional View): This diagram illustrates the primary ways the End User and Administrator interact with the system's features.



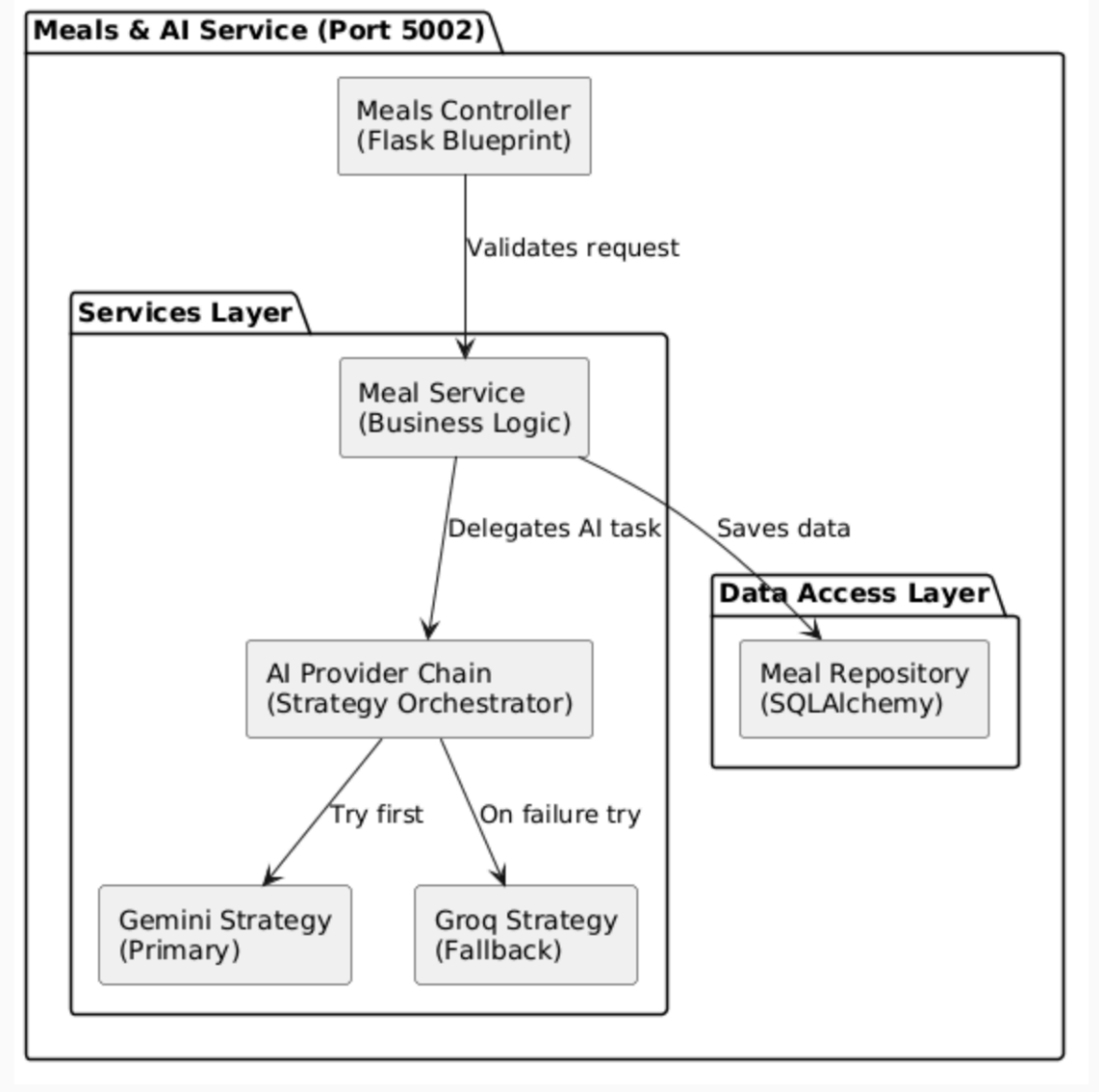
Context Diagram (Context View): This diagram treats the StayFit backend as a single black box, highlighting its external boundaries and integrations with third-party providers.



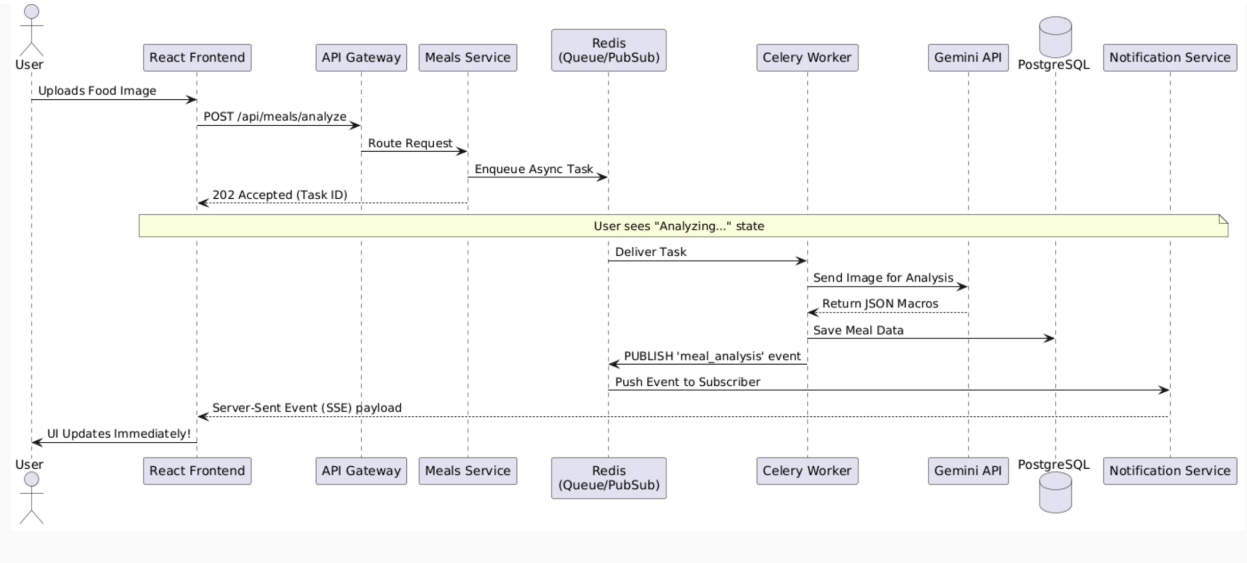
Container Diagram (Deployment View): Zooming into the system, this diagram shows the highly-available Microservices architecture orchestration. It illustrates how the API Gateway routes requests to independent processes and background queues.


End User
HTTPS / SSE
StayFit Microservices Architecture
React Frontend
(PWA)
REST API Calls
API Gateway
(Flask, Port 5000)
Meals & AI Service
(Port 5002)
Analytics Service
(Port 5003)
Auth Service
(Port 5001)
Enqueue Job
Cache Aside
Cache Aside
Redis
(Broker, Cache, Pub/Sub)
Consume Job
Publish Event
Subscribe Event
Celery Workers
(Async AI Processing)
Notifications Service
(Port 5004)
Save Analysis
PostgreSQL
(Persistent Storage)

Component Diagram (Module View - Meals & AI Service): Zooming into the "Meals & AI Service" container, this diagram shows the internal layered MVC structure and the implementation of the AI Strategy Pattern.



Sequence Diagram (Runtime View - Core Meal Flow): This diagram illustrates the complex asynchronous flow of data from the moment a user uploads a photo, mapping exactly how Redis and Celery process the task without blocking the web server.



Part 2: Architecture Decision Records (ADRs)

Following the Michael Nygard format, these records capture the most significant architectural choices, the tension of the forces that drove them, and the long-term consequences on the project.

ADR 1: Use Flask for Backend API Framework

Context: The team requires a backend framework to serve the REST API. The system will heavily rely on AI integrations, making Python the optimal language due to first-class SDK support from Google and Groq. The team evaluated Django, FastAPI, and Flask. Django was deemed too monolithic ("batteries-included") for an API-only backend that relies heavily on a decoupled React frontend. FastAPI offers native async support, but the team's familiarity with traditional WSGI patterns and the need for extreme flexibility in architectural layout created tension.

Decision: We will use **Flask** as the core API framework, utilizing `Flask-JWT-Extended` for authentication and `SQLAlchemy` for ORM.

Status: Accepted.

Consequences

- **Positive:** Flask's micro-framework nature allowed us to strictly implement our own Module View (separating `routes`, `services`, and `repositories`) without fighting the framework's default folder structures. Using Blueprints made the routing highly modular.
- **Negative:** Because Flask is fundamentally a synchronous WSGI framework, we cannot use modern Python `async/await` syntax natively for our I/O bound network calls. This forces our hand in ADR 2 (requiring external task queues) to handle the long-running AI API requests.
- **Neutral:** Development speed was very high due to ecosystem maturity, but we had to manually wire up structural patterns that a heavier framework might have provided out of the box.

ADR 2: Use Celery for Asynchronous AI Analysis

Context: The core user flow involves taking a photo of food and receiving nutritional data. Transmitting the image, processing it through a vision model (Gemini), and parsing the results takes between 5 and 15 seconds depending on network conditions and API load. If this work is done synchronously in the Flask route, the HTTP request hangs. If a dozen users log meals simultaneously, the entire Flask connection pool is exhausted, and the application crashes for everyone. We must balance the need for immediate UI feedback with the reality of slow external network calls.

Decision: We will use **Celery** as a distributed task queue, backed by **Redis** as the message broker, to offload all AI API interactions and image processing to asynchronous background workers.

Status: Accepted.

Consequences

- **Positive:** The Usability and Performance concerns are fully addressed. The Flask API responds to image uploads in milliseconds, allowing the React frontend to show an optimistic "Analyzing..." state. The system is highly scalable; we can spin up dozens of Celery workers specifically dedicated to AI processing without touching the web servers.
- **Negative:** Significant architectural overhead. We must now deploy, monitor, and secure a Redis instance. We must manage Celery worker lifecycles. We also introduced the complex problem of communicating the task's completion *back* to the user (addressed in ADR 3).
- **Technical Debt:** Error handling becomes harder. If an AI call fails in a background worker, we must manually construct pathways to inform the frontend of the failure.

ADR 3: Use Pub/Sub Notification System via Redis

Context: When a user logs a meal that pushes their daily intake over 80% of their calorie budget, or when an AI analysis completes asynchronously, the system needs to immediately alert the user in real-time. Traditional approaches involve having the frontend incessantly poll the backend database for new records, which wastes network bandwidth and database connections. We need a performant push-notification mechanism. To consolidate our messaging infrastructure efficiently alongside our asynchronous task queue and in-memory caching layer, we chose to utilize Redis.

Decision: We decided to implement a **Publish-Subscribe (Pub/Sub) Pattern** utilizing **Redis** as a centralized, high-performance message broker and **Server-Sent Events (SSE)** for the frontend connection. A dedicated streaming microservice subscribes to Redis channels and yields formatted Server-Sent Events directly to the frontend the exact microsecond a payload arrives.

Status: Accepted.

Consequences

- **Positive:** Highly efficient, real-time updates. Postgres is entirely alleviated from maintaining long-lived idle connections for thousands of listening clients. Redis Pub/Sub operates exclusively in-memory, handling throughput magnitudes faster than disk-based event triggers.
- **Negative:** Redis Pub/Sub is inherently "fire-and-forget". If the Notification subscriber or the user's browser SSE stream disconnects exactly as a payload is published, that notification is permanently lost. SSE is also strictly unidirectional (server-to-client only).

ADR 4: Use Strategy Pattern for AI Provider Fallback

Context: The entire value of the application hinges on the AI's ability to analyze food. Google's Gemini API is highly capable but is subject to aggressive rate limiting on free/standard tiers, and like any cloud service, is susceptible to outages. Hardcoding Gemini SDK calls directly into the application logic introduces a catastrophic single point of failure.

Decision: We will implement the **Strategy Design Pattern** for all AI interactions. We define an abstract base class `AIProviderStrategy` with methods for `analyze_vision` and `analyze_text`. We implement a concrete `GeminiStrategy` (Primary) and a `GroqStrategy` (Fallback, utilizing Llama models).
A `ProviderChain` orchestrator will attempt the primary strategy, catch any exceptions (timeouts, rate limits), and automatically execute the fallback strategy.

Status: Accepted.

Consequences

- **Positive:** Massive boost to the Availability and Fault Tolerance concerns. The system is highly resilient; end-users will simply experience a slightly different AI analysis rather than an application crash if Google's servers go down. It also satisfies the Open/Closed Principle—if we wish to add OpenAI (ChatGPT) in the future, we simply create an `OpenAIStrategy` class without modifying any existing meal logging logic.
- **Negative:** Maintenance overhead is increased. Because different LLMs require different prompt tuning and return slightly different JSON flavors, we must maintain robust parsing and cleaning logic (`_clean_json_response`) to ensure both Gemini and Groq outputs map cleanly to our Postgres schema. We effectively have to write tests and prompts twice.

ADR 5: Use Microservices Architecture via API Gateway

Context: As the project grows, especially with the integration of heavy AI processing for meal analysis alongside standard CRUD operations and real-time analytics, running everything in a single process introduces risks. For example, a heavy load on the Gemini AI image parsing could crash or stall the entire application, preventing users from accessing their analytics or authenticating.

Decision: We elected to partition the application across disparate network boundaries using a **Microservices Architecture**, fronted by an API Gateway (`start_microservices.py`). We separated the application into domains (API Gateway, Auth, Meals & AI, Analytics, Notifications) operating as independent Flask processes.

Status: Accepted.

Consequences

- **Positive:** Enables independent horizontal scaling and fault isolation. If the Meals service crashes due to heavy AI image processing loads, users can still view their Analytics charts natively without interruption.
- **Negative:** Increased deployment and operational complexity. Running and managing multiple processes is more complex than a single monolithic app.

ADR 6: Use Federated Identity Pattern (Google Auth)

Context: User onboarding is a critical funnel for health tracking applications. Requiring users to fill out long registration forms and manage passwords introduces friction. Furthermore, storing and securing user passwords manually places the burden of security, hashing logic, and credential management on our application, increasing the risk profile.

Decision: We decided to implement the **Federated Identity Pattern** using Google OAuth2. When a user logs in via Google, an encrypted JWT is verified securely by the backend. If the user is new, a `User` record is automatically registered using their Google email. Ultimately, the backend then generates a distinct system-internal JWT access token (`flask_jwt_extended`), decoupling the internal session lifecycle from Google.

Status: Accepted.

Consequences

- **Positive:** Zero-Trust Security (we offload password security to Google) and reduced onboarding friction, maximizing sign-up conversion.
- **Negative:** We are reliant on Google's OAuth infrastructure. If Google Auth goes down or changes its API, our authentication flow could be impacted. It also requires maintaining client IDs across environments.

ADR 7: Use Redis Caching Layer (Cache-Aside Pattern)

Context: As the application scales, expensive read-heavy operations emerged as system bottlenecks. Generating personalized AI meal suggestions dynamically consumes billable external tokens even when users share identical dietary constraints. Furthermore, heavy iterative database queries for analytics sum multiple days of logs, unnecessarily straining the PostgreSQL database on every page navigation.

Decision: We elected to introduce an in-memory **Redis Caching Layer** adopting the **Cache-Aside** architectural pattern via `cache_service.py`. We cache AI generation keys based on semantically aligned profile attributes (Global Semantic Caching) and use a reusable `@cache_response(timeout=300)` decorator structurally attached to expensive reporting routes (User-Scoped Route Caching).

Status: Accepted.

Consequences

- **Positive:** Drastically reduced latency (complex aggregations load in < 5ms). It spares the PostgreSQL DB from repetitive sequential scans and protects our external AI API billing thresholds. The graceful degradation fallback pattern ensures that a Redis disconnection does not halt system transactions.
- **Negative:** Stale data risks. Users accessing analytics might view slightly older progression stats within the short lifespan (5-minute TTL) window before the cache expires. Adds a dependency (Redis) that must be strictly monitored.

Task 3: Architectural Tactics and Patterns

This document details the specific architectural tactics and design patterns employed within the StayFit application. The terminology and conceptual framework heavily reference standard Software Engineering principles (such as those outlined by Len Bass regarding System Qualities: Availability, Performance, Modifiability, Testability, Usability, and Security).

Part 1: Architectural Tactics

Architectural tactics are design decisions used to achieve a desired quality attribute response. The following 5 tactics have been implemented in the codebase to satisfy specific Non-Functional Requirements (NFRs).

1. Security Tactics

1.1. Authentication via OAuth (Google)

Non-functional Requirements Addressed: Security, Confidentiality, Usability

Implementing OAuth integration with Google delegates user authentication to a trusted external provider. This eliminates the need to store sensitive password data internally while providing a familiar, secure, and streamlined login experience for users.

1.2. Authorization via Role-Based Access Control (RBAC) & JWT Isolation

Non-functional Requirements Addressed: Access Control, Privacy

Our system enforces data privacy by utilizing `Flask-JWT-Extended` for stateless session management. Access control checks are embedded at the API layer, ensuring that user records and daily logs are strictly isolated by `user_id` at the repository level. This structured authorization minimizes the risk of unauthorized data access (IDOR attacks).

2. Availability Tactics

2.1. Fault Masking via Active Redundancy / Fallback

Non-functional Requirements Addressed: System Uptime, Fault Tolerance

Relying solely on a single AI provider (Google Gemini) introduces a single point of failure. The application anticipates external AI API timeouts or rate limits and seamlessly switches to a backup component (`GroqStrategy`) without the user experiencing a system crash.

3. Modifiability Tactics

3.1. MCS Pattern Implementation -> Modifiability

Non-functional Requirements Addressed: Adaptability, Maintainability

The Models-Controllers-Services (MCS) pattern (implemented via Flask Blueprints, Services, and Repositories) separates data (Model), presentation (React UI), and business logic (Services/Controllers), allowing independent development, testing, and maintenance. This separation minimizes coupling; a change to the database engine does not require a rewrite of the business rules.

4. Usability Tactics

4.1. System Initiative via Server-Sent Events (SSE)

Non-functional Requirements Addressed: Usability, System Feedback

Because meal analysis happens asynchronously via background queues, the UI must update without requiring a manual refresh. The system takes the initiative to push real-time notifications via Server-Sent Events, providing immediate feedback on background task completion, maximizing user confidence.

5. Scalability Tactics

5.1. Microservices Architecture

Non-functional Requirements Addressed: Scalability, Resilience

By decomposing the monolithic application into independent microservices (Auth, Meals & AI, Analytics, Notifications) via an API Gateway, we enable the individual scaling of components based on demand. This isolation improves overall fault tolerance—if the AI parsing service crashes under load, the Authentication and Analytics services remain unaffected.

5.2. Message Broker (Redis)

Non-functional Requirements Addressed: Traffic Management, Scalability, Decoupled Communication

Implementing Redis for asynchronous task queuing (via Celery) and Pub/Sub messaging decouples service communications. This approach buffers heavy background workloads (like AI vision processing) to prevent synchronous overload on the Flask HTTP connection pool, ensuring the API remains highly responsive.

Part 2: Implementation Patterns

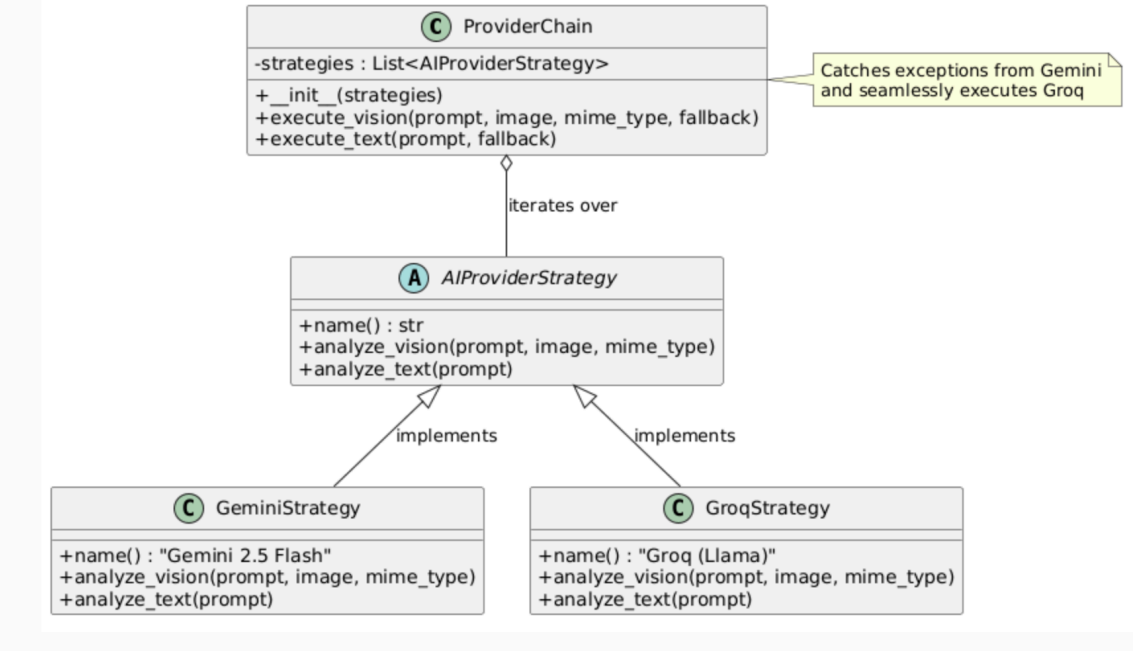
1. Strategy Pattern (Low-level Design Pattern):

The Strategy pattern lets you define a family of algorithms, encapsulate each one, and make them interchangeable. In StayFit, this pattern is deployed to ensure maximum availability of the core AI meal logging feature by allowing seamless swapping between different Large Language Models (LLMs).

Implementation Details:

- **Interface:** Located in `gemini_service.py`, the abstract base class `AIProviderStrategy` defines the contract for `analyze_vision` and `analyze_text`.
- **Concrete Strategies:** Implementations `GeminiStrategy` (Primary) and `GroqStrategy` (Fallback) handle specific SDKs and JSON formatting.
- **Orchestrator:** The `ProviderChain` class iterates over the strategies. If the primary catches an exception (rate limit/timeout), it automatically executes the exact same prompt against the fallback strategy.

Benefits: Massive boost to Availability and Fault Tolerance. The system satisfies the Open/Closed Principle—allowing future additions like OpenAI without modifying existing meal logging logic.



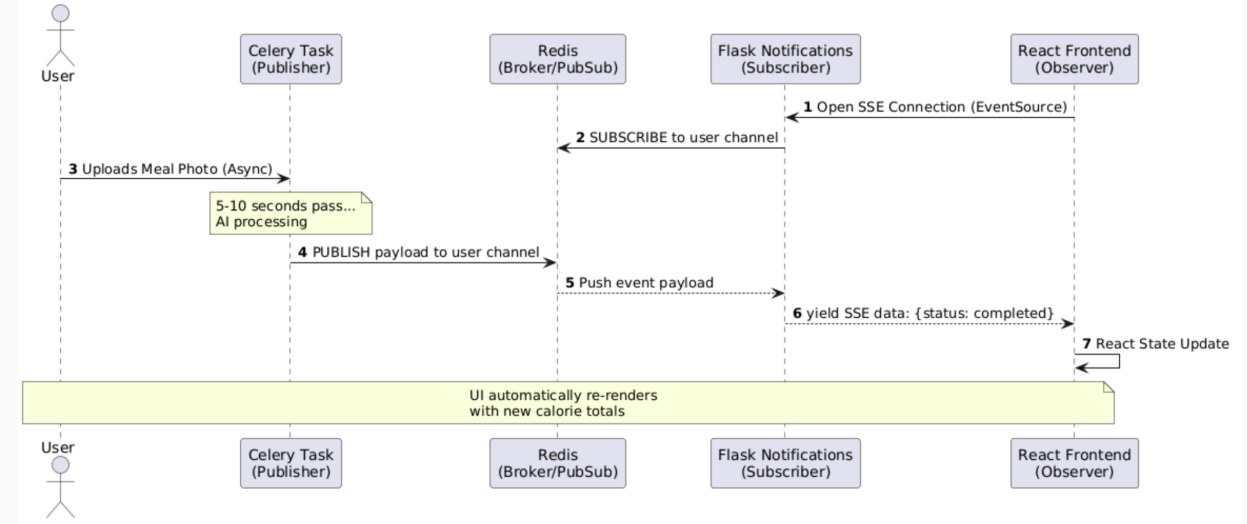
2. Publish-Subscribe with Redis (Architectural Design Pattern):

The system adopts a publish-subscribe (Pub/Sub) architecture using Redis as the central message broker for decoupled, event-driven communication to solve the problem of real-time asynchronous task completion updates.

Message Flow:

1. **Publisher:** The Celery background worker successfully parses an image, saves the meal to the Postgres database, and publishes an event payload to a Redis channel (e.g., `user_notifications`).
2. **Broker:** Redis immediately pushes the message to all active subscribers.
3. **Subscriber:** The Flask Notification service (`notifications.py`) subscribes to Redis and catches the event, formatting it into a Server-Sent Event (SSE) stream.
4. **Client:** The React frontend's `EventSource` listens to this SSE stream and automatically updates the `MealContext` UI state.

Benefits: Loose coupling between the background workers and the frontend; Highly efficient real-time updates without polling the database; Scalable and fast because Redis operates exclusively in-memory, relieving Postgres of long-lived connection strain.



Task 4: Prototype Implementation and Analysis

Prototype Development

Github repository link: https://github.com/paridhii16/SE-Project-3.git

We have developed a working prototype of the StayFit system that demonstrates the core functionality and architectural design. The prototype focuses heavily on the end-to-end meal logging and analysis pipeline, utilizing external AI providers and asynchronous background processing. This reflects the practical application of our proposed microservices solution.

Architecture Analysis: Microservices vs Monolithic

This report compares two architectural patterns for handling the StayFit backend infrastructure: the implemented **Microservices Architecture** and an alternative **Monolithic Architecture**. The goal is to evaluate trade-offs for non-functional requirements such as response time, availability, scalability, maintainability, and deployment complexity.

- **Implemented Architecture:** Microservices with API Gateway. StayFit currently runs as a set of Flask-based services (Auth, Meals, Analytics, Notifications) behind an API Gateway on port `5000`.
- **Alternative Architecture:** Monolithic Flask Application. All backend domains share the exact same runtime process (`run.py` on port `5000`).

Aspect	Implemented Architecture (Microservices)	Alternative Architecture (Monolith)
Communication Model	API Gateway routing to decoupled REST services	In-process function calls
Technology Used	Flask, Redis, Celery, Multiprocessing	Flask

Aspect	Implemented Architecture (Microservices)	Alternative Architecture (Monolith)
Message Flow	Frontend → Gateway → Specific Domain Service → DB/Redis	Frontend → Monolithic Backend -> DB
Triggering Mechanism	Network routing via Gateway	Internal WSGI routing
Dependencies	Requires Redis broker, Celery workers, and multiple open ports	Single process, only requires DB

Non-Functional Requirements Evaluated

- **Response Time:** Evaluated using average, p50, and p95 latency for selected authenticated API endpoints (Daily/Weekly Analytics, Meal Logs, Auth Profile).
- **Availability / Fault Isolation:** Evaluated by forcefully stopping the Meals/AI service and checking the availability of unaffected domains.

Comparison

a. Response Time

Note: Throughput and latency measured using `scripts/benchmark_architecture.py` over 100 authenticated requests per endpoint.

Architecture	Average Response Time	Throughput (req/s)	Description
Monolith	5.58 ms	798.30	Lower average response times and higher throughput due to zero network routing overhead.
Microservices	7.86 ms	543.09	Slightly higher latency due to API Gateway proxy routing, but perfectly acceptable for UI responsiveness.

Outcome: The monolithic architecture shows lower average response times and higher throughput for basic endpoints because it avoids the internal network hops of an API Gateway. However, the microservices architecture still provides fantastic performance (< 10ms average latency) that easily satisfies UI responsiveness requirements.

b. Availability and Fault Isolation

Note: Evaluated by simulating a crash in the Meals/AI processing domain (Port 5002) and measuring gateway response codes.

Architecture	Unavailable Domains	Still Available Domains	Description
Microservices	2 of 4	2 of 4	Partial service remains available. Analytics and Auth stay online even if Meals crashes.
Monolith	4 of 4	0 of 4	Full backend outage. A crash in the AI processor brings down the entire application.

Outcome: Microservices provide vastly superior fault isolation. During our benchmark test with the Meals service stopped, `/api/meals/today` returned HTTP 503 for all requests, while analytics and auth endpoints remained perfectly available and returned HTTP 200. This demonstrates effective fault isolation: a failure of the heavy AI service does not impact the availability of the dashboard.

Trade-Off Analysis

Trade-Off	Microservices with Gateway	Monolithic
Scalability	Meals/AI service and Celery workers can be scaled independently from lightweight Auth.	Requires replicating the whole backend even if only AI needs more resources.
Availability	Better fault isolation. A Meals service failure does not automatically break Auth or Analytics.	Lower fault isolation. A backend crash removes all API functionality.
Performance	Slight proxy overhead from gateway routing.	Fewer network hops for normal routes.
Maintainability	Clearer domain boundaries. Enforces strict modularity between Auth, Meals, etc.	Domains become more tightly coupled and tangled as the app grows.
Operational Complexity	Requires gateway, multiple Flask services, Celery, Redis. High deployment overhead.	One backend process. Easier local startup and simpler debugging.

Conclusion: For an early prototype, a monolithic Flask application is definitely easier to run and debug locally. However, StayFit has workloads with vastly different failure and scaling characteristics. AI meal analysis is slow and provider-dependent, analytics is read-heavy but predictable, and auth is security-sensitive.

The implemented microservices architecture is therefore entirely justified. The API gateway adds minimal latency (~2ms) but preserves a single frontend API contract while allowing independent service failure and future independent scaling. The monolith is simpler for initial development, but the microservices design successfully supports StayFit's AI-heavy and growth-oriented requirements.

Reflections and Lessons Learned

- Working on this project taught us a tremendous amount about building scalable and modular systems. Designing with microservices helped us clearly separate concerns, making the system easier to extend and maintain without stepping on each other's code.
- One of the key lessons was how important it is to think about non-functional requirements (like scalability and availability) early in the design process. The realization that slow AI processing could freeze our entire web server drove the adoption of Celery and Redis Pub/Sub. These early considerations shaped many of our architectural decisions and ensured the system could handle real-world latency from external APIs.
- We also realized how important clear communication and strict API contracts are, especially when working on different services that depend on each other via an API Gateway. Finally, the use of design patterns like Strategy (for AI fallback) and Proxy (for database caching) proved invaluable in masking third-party API failures and optimizing database loads.

Contributions

Everyone contributed equally to the project.